REST API Fundamentals

learn more

Testleaf

Always Ahead

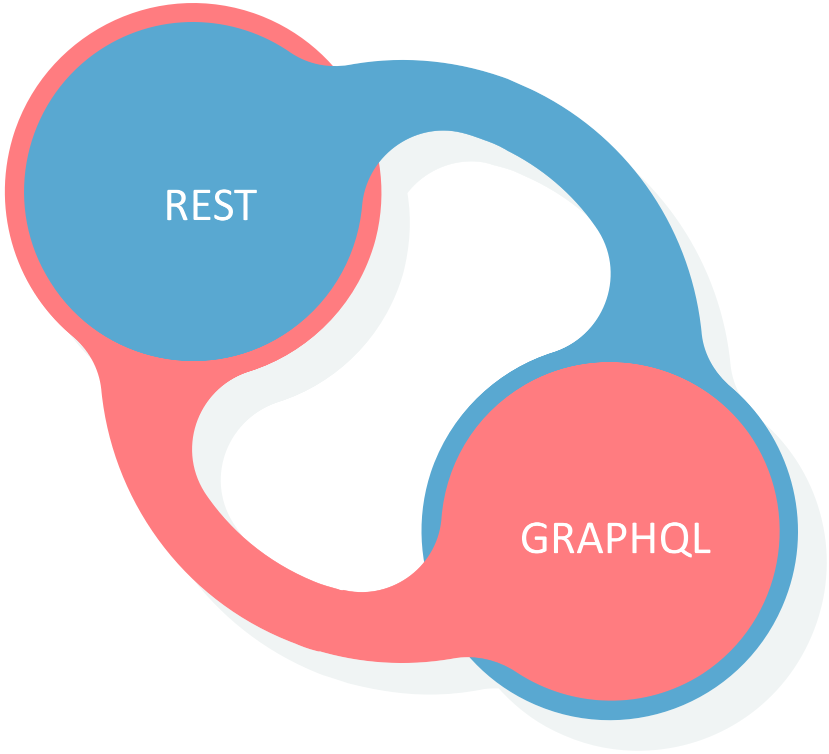
Agenda for today

1. Quick overview to the full program
2. Why API Automation is needed today?
3. Different Protocols and Architectures
4. RESTful API Fundamentals
5. Steps to test / automate the API

2 API Architectures

1 REST

Representational state transfer (**REST**) is a software architectural style



GRAPHQL 2

GraphQL is an open-source data query and manipulation language for APIs

Tools/Libraries that you will be learning

REST

REST

REST

1. **PostMan**

   a collaboration platform for API development.

2. **RestAssured + WireMock**

   Java DSL framework for simplifying testing of REST based services

3. **Swagger**

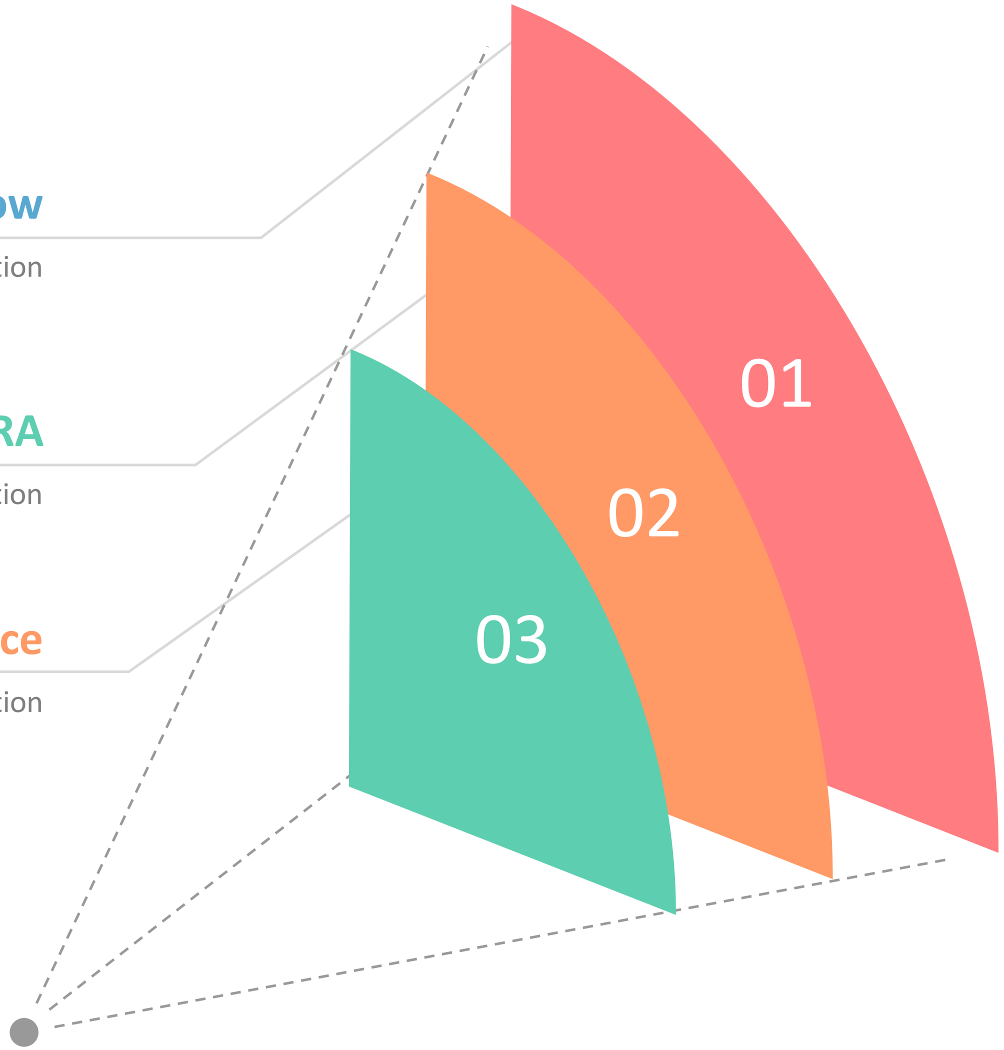
   Framework backed by a large ecosystem of tools for documentation

Applications Under Test (AUT)

ServiceNow
ITSM Application

JIRA
Project Management Application

Salesforce
CRM Application



Customized Abstracts - Frameworks

01

Matschie

REST Assured
+
Cucumber
+
Allure

02

Makaia

Rest Assured
+
Selenium
+
Cucumber
+
Extent

Why API Automation is growing in demand?

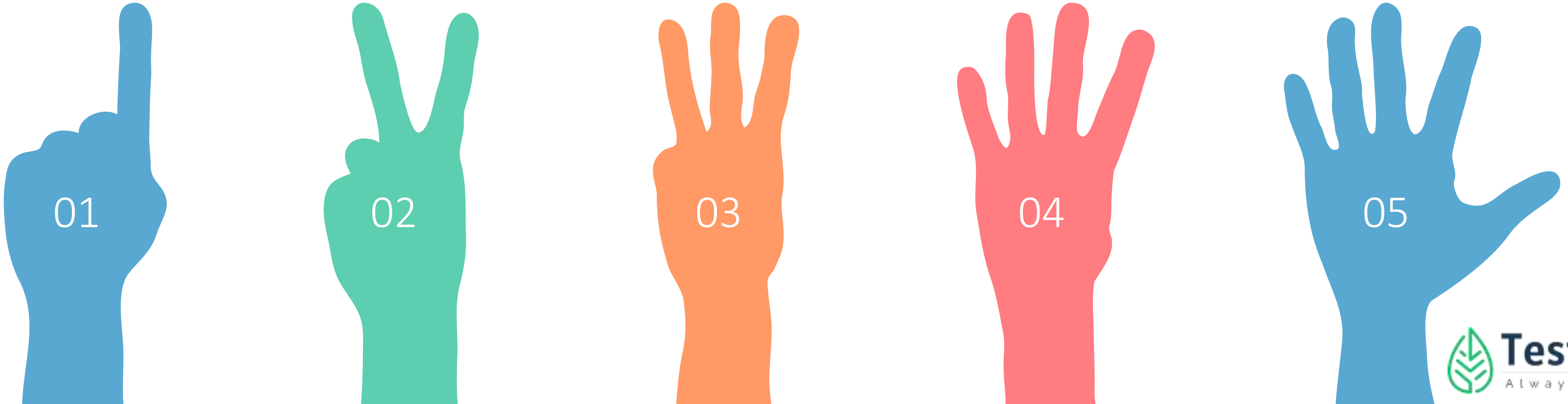
API Based Apps

Faster

Easier & Isolated

Data Generation

Less Failures



Why API Automation is growing in demand?

API Based Apps

Testleaf
Always Ahead

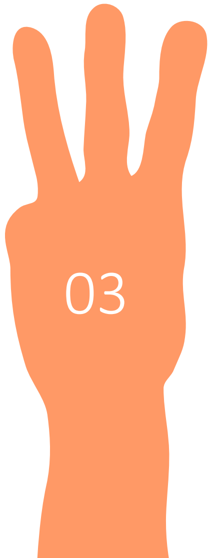
Why API Automation is growing in demand?

Faster

Rendering Engine

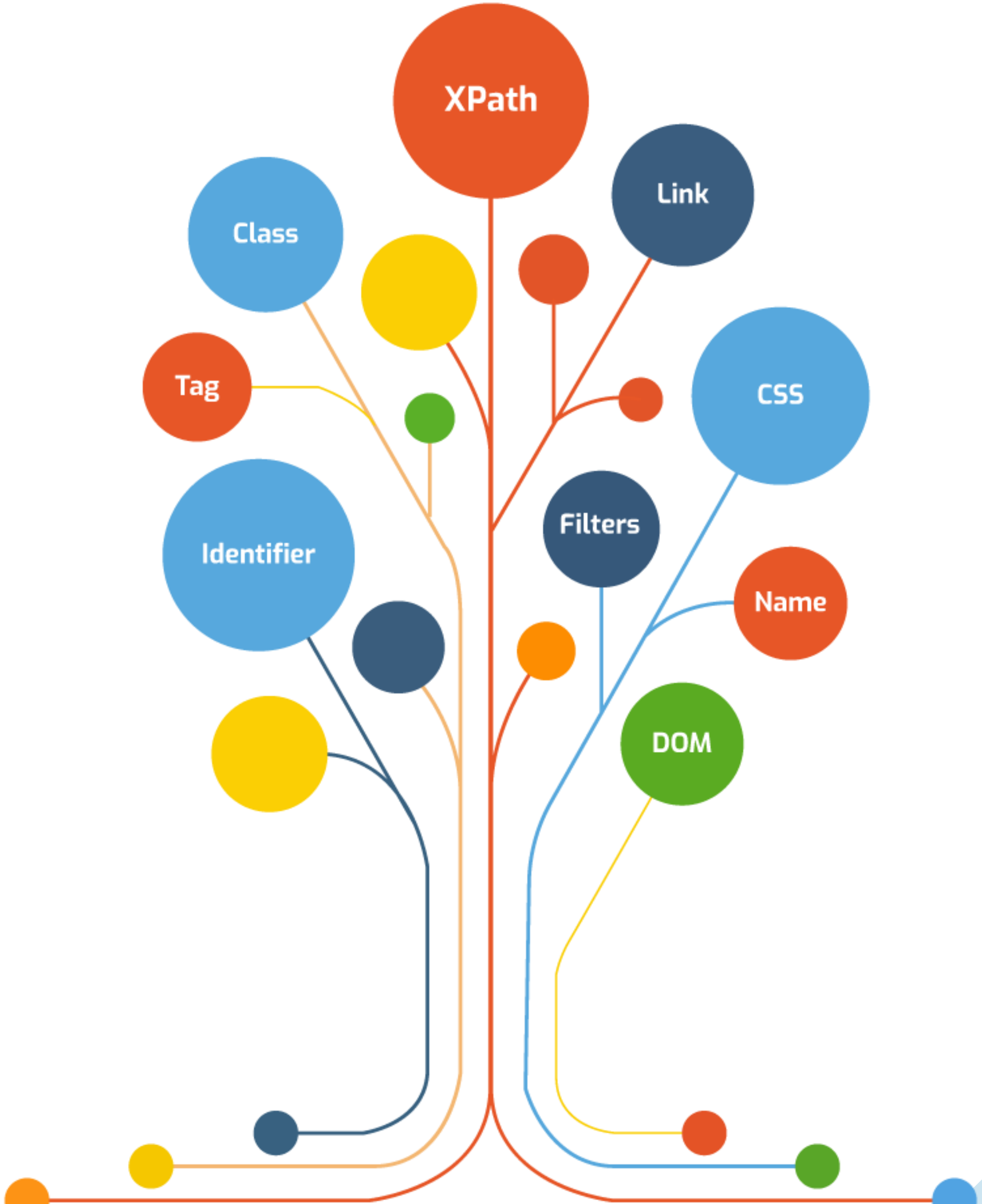
Why API Automation is growing in demand?

Easier & Isolated



Browser Specific

Element Challenges



Why API Automation is growing in demand?

Data Generation

Why API Automation is growing in demand?

Less Failures

Testleaf
Always Ahead

Element Not Found Challenges

Time Out (Behavioral) Failures

Summary !!

01 API Based Apps

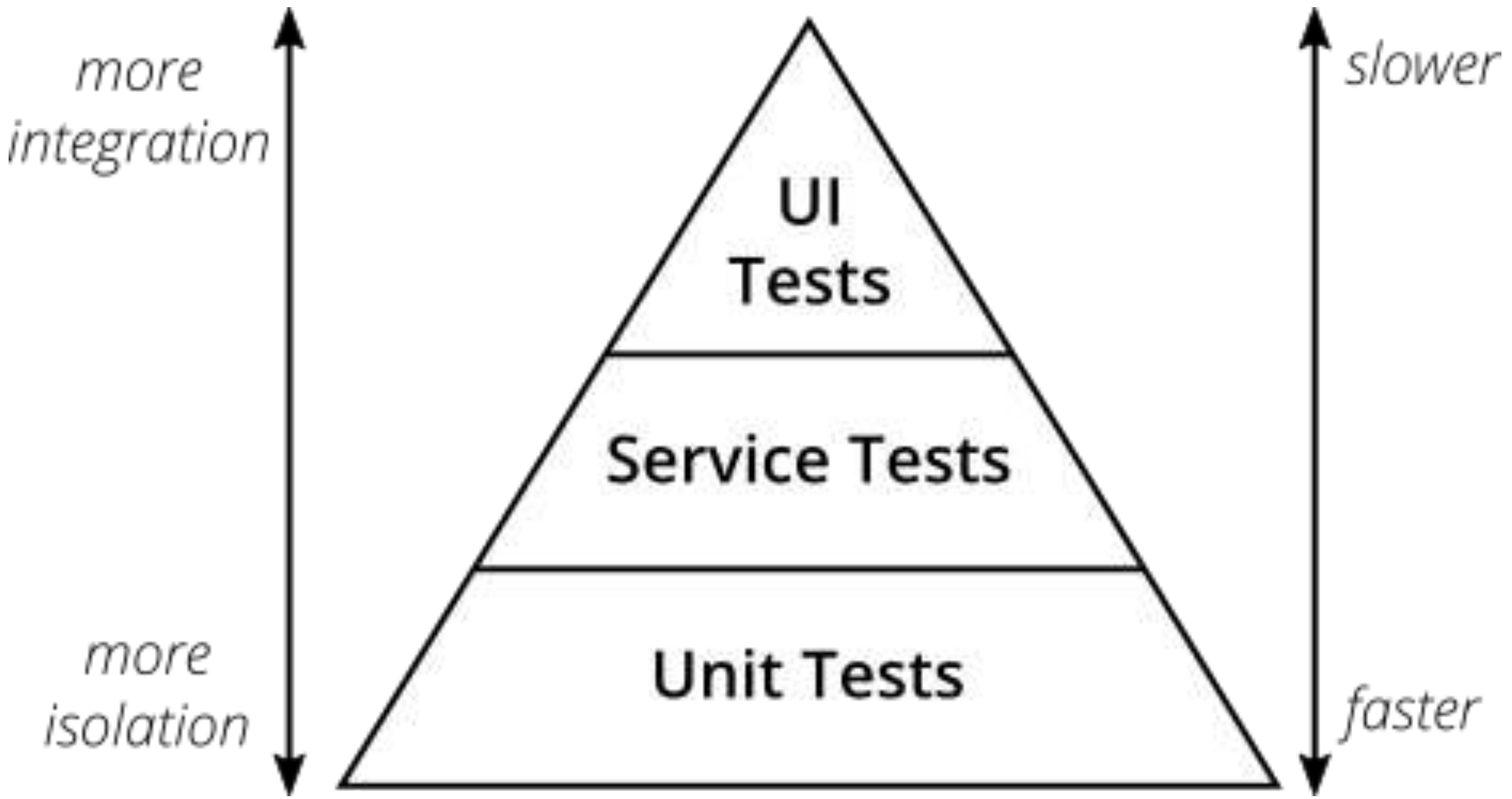
02 Faster

03 Easier & Isolated

04 Data Generation

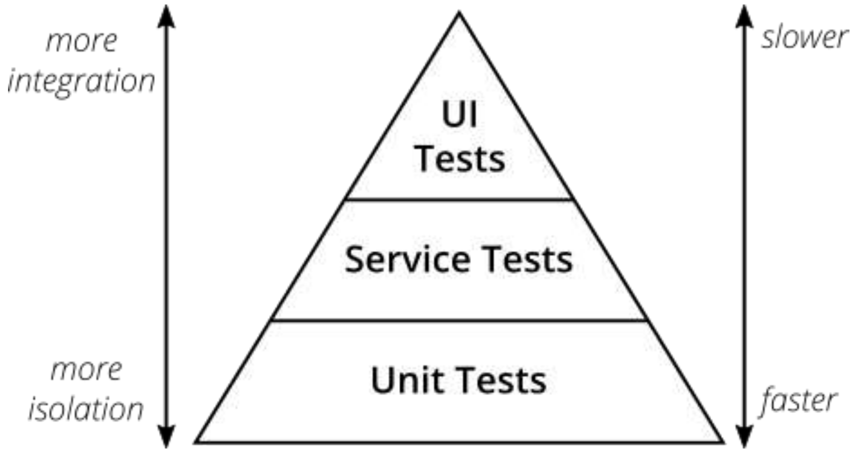
05 Less Failures

Mike Cohn’ Test Pyramid



Test Pyramid - Take Away

1. Write tests with different granularity (End to End tests in UI)
2. The more high-level you get the fewer tests you should have



What is API?

What is API?

- API stands for **A**pplication **P**rogramming **I**nterface

- API is a set of rules and protocols that enables the communication between different software applications.

- The simple real-time analogy to understand what an API is “Customer-Waiter-Chef” analogy.

Testleaf
Always Ahead

Types of Web APIs

1. RESTful API (REpresentational State Transfer)
2. SOAP API (Simple Object Access Protocol)
3. GraphQL
4. JSON – RPC and XML – RPC
5. OData (Open Data Protocol)
6. gRPC (Remote Procedure Calls)

SOAP vs REST vs GraphQL

SOAP - Simple Object Access Protocol (1998)

REST - Representational State Transfer (2000)

GraphQL - Graphical Query Language (2015)

SOAP

SOAP (Simple Object Access Protocol) is a messaging protocol specification for exchanging structured information in the implementation of Web Services.

Element	Description	Required
Envelope	Identifies the XML document as a SOAP message.	Yes
Header	Contains header information.	No
Body	Contains call and response information.	Yes
Fault	Provides information about errors that occurred while processing the message.	No

- 1998
- XML
- GET, POST
- Higher Payload
- No Cache

REST

Representational state transfer (REST) is a software architectural style that defines a set of constraints to be used for creating Web Services

HTTP METHODS	Collection resource, such as `https://api.example.com/collection/`
GET	*Retrieve* the URIs of the member resources of the collection resource in the response body.
POST	*Create* a member resource in the collection resource using the instructions in the request body. The URI of the created member resource is *automatically assigned* and returned in the response *Location* header field.
PUT	*Replace* all the representations of the member resources of the collection resource with the representation in the request body, or *create* the collection resource if it does not exist.
PATCH	*Update* all the representations of the member resources of the collection resource using the instructions in the request body, or *may create* the collection resource if it does not exist.
DELETE	*Delete* all the representations of the member resources of the collection resource.

- 2000
- XML, JSON, TXT, HTML
- GET, POST, DELETE PUT, PATCH, HEAD ..
- Lower Payload
- Stateless, Cache

GraphQL

GraphQL is an open-source data query and manipulation language for APIs, and a runtime for fulfilling queries with existing data

- 2015
- JSON
- Asynchronous
- HTTP, AMQP, MQTT
- Send Only required data

Testleaf
Always Ahead

HTTP Methods (CRUD)

1

GET

Retrieve resource information (READ)

2

POST

Create new subordinate resources (CREATE)

3

DELETE

delete resources (DELETE)

4

PUT

primarily to update existing resource (REPLACE)

5

PATCH

make partial update on a resource (UPDATE)

Authentication

No Authentication

Examples: Weather Forecast, Google Map

basic.

Basic

Examples: ServiceNow, JIRA

OAuth 2.0

Examples: Payment Gateways, ServiceNow

Token

Examples: Oracle WMS, JWT

HTTP Headers

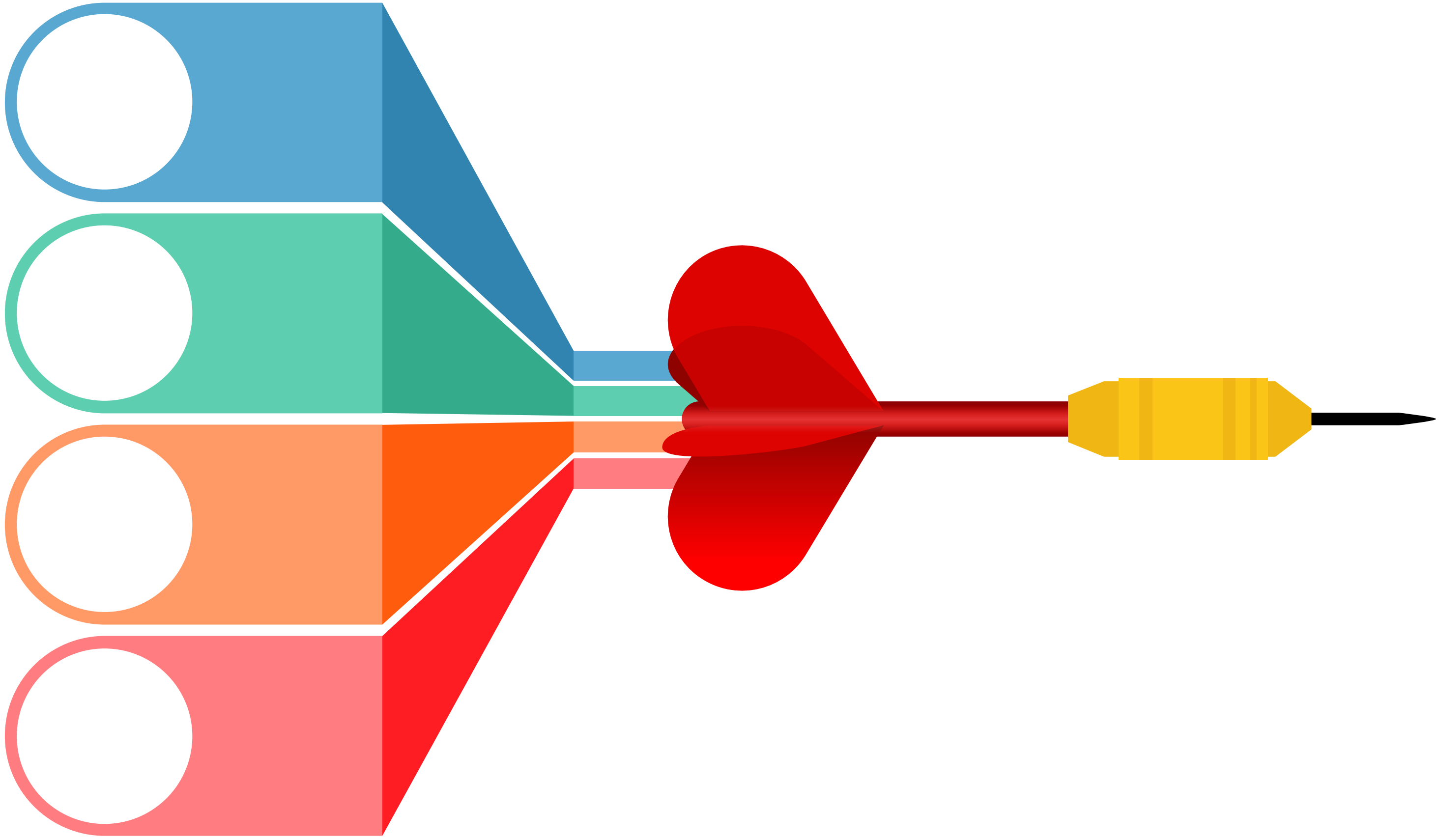
Content-Type

The Content-Type entity-header field indicates the media type of the entity-body sent to the recipient. **(request)**

Accept

The Accept request-header field can be used to specify certain media types which are acceptable for the **response**.

Request Parameters



Header

Request header, usually related to authorization.

Query

Parameters in the query string of the endpoint, after the ?.

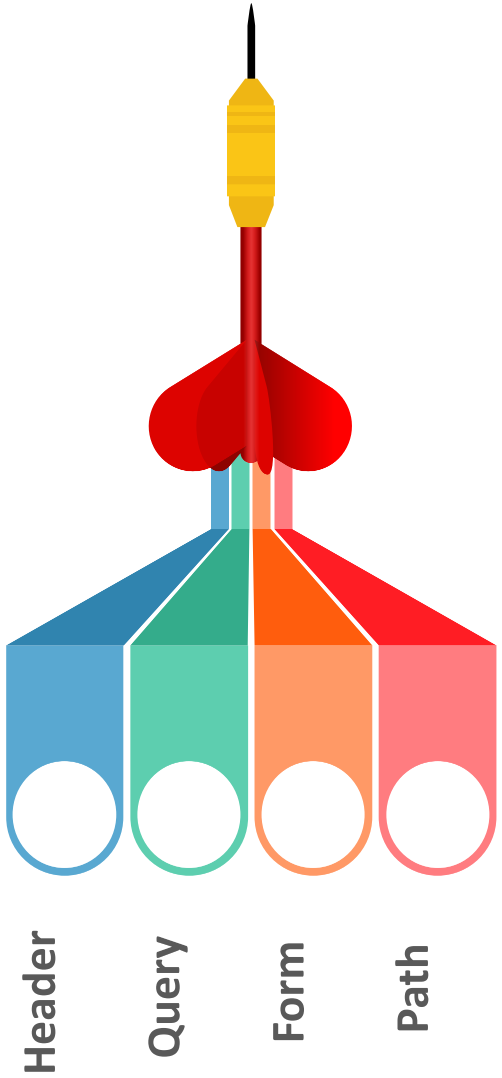
Form / Request Body

Parameters included in the request body. Usually submitted as JSON.

Path

Parameters within the path of the endpoint, before the query string (?). These are usually set off within curly braces.

Request Parameters – Data Types



These data types are the most common with REST APIs:

- **string**: An alphanumeric sequence of letters and/or numbers
- **integer**: A whole number – can be positive or negative
- **boolean**: true or false value
- **object**: Key-value pairs in JSON format
- **array**: A list of values

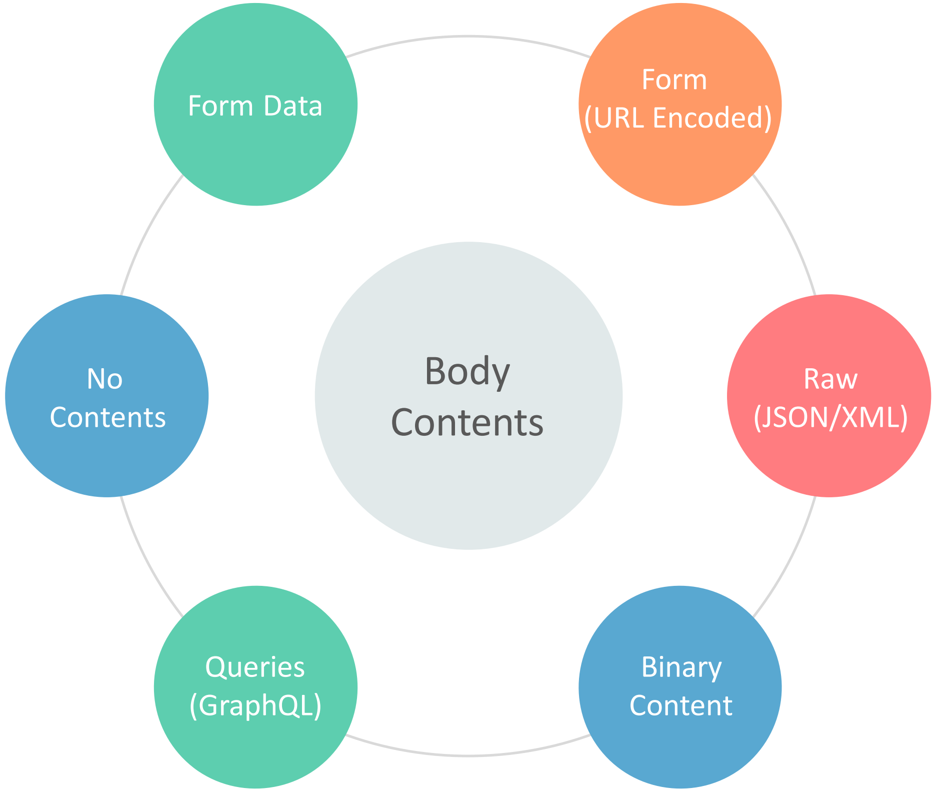
Request Body (Data Formats)

Form Data

In Key-value format (Text / File) including multi-part

Form (URL Encoded)

Data is encoded in key-value format



None

No data passed in body

Raw Data

This can be JSON/ XML or TXT

Queries & Variables

This is specially for the GraphQL Scenarios

Binary

The files or executables that can be attached

HTTP Response

HTTP Response has 4 components:

Response Status Code: This represents the server response status code for the requested resource. Example:- 4xx represents client error, 2xx represents a successful response.

HTTP Version: Indicates the HTTP protocol version.

Response Header: This part has the metadata of the response message. Data can describe what is the content length, content type, response data, what’s server type, etc.

Response Body: This part contains what’s the actual resource/message returned from the server.

Status Codes

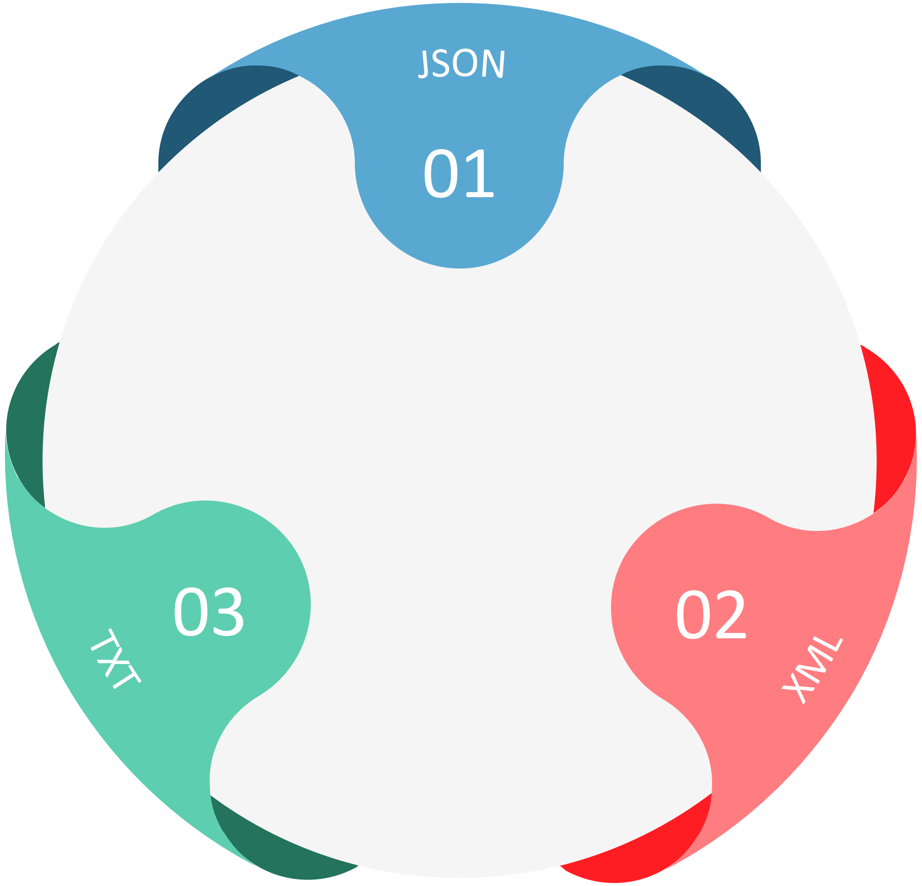
100 -199	Information
200 - 299	Success
300 - 399	Redirects
400 - 499	Clients Errors
500 - 599	Server Errors

Testleaf
Always Ahead

Body (Contents)

The body can have different formats like below:

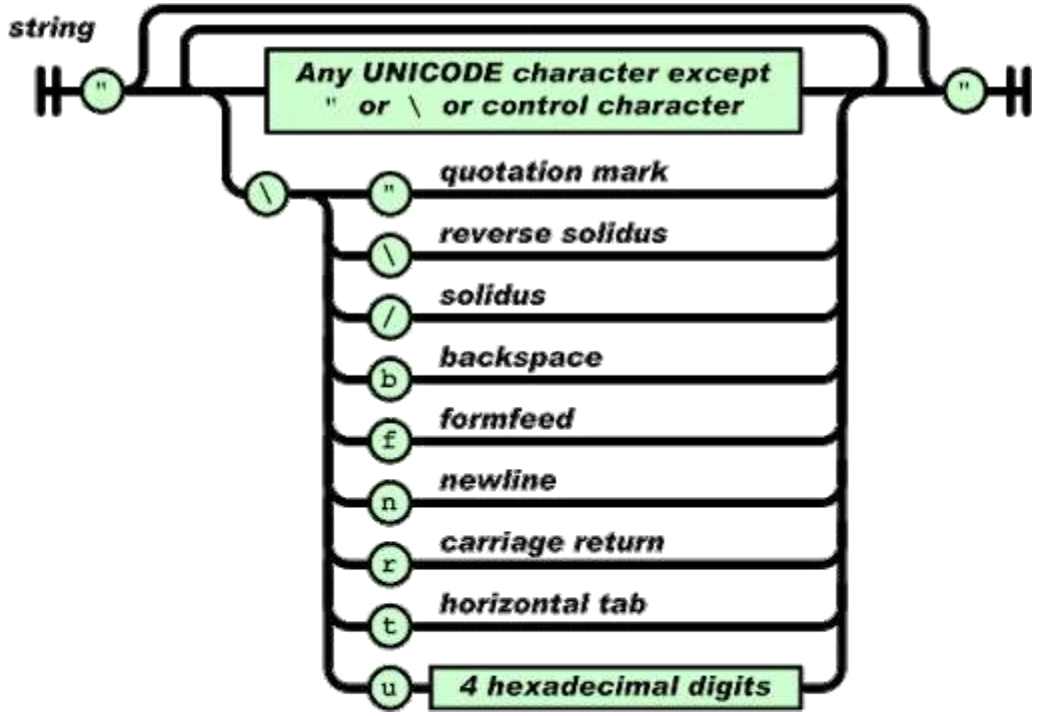
- JSON - JavaScript Object Notation
- XML - Extensible Markup Language
- TXT – Plain text format



Testleaf
Always Ahead

JSON – A quick run through

- JavaScript Object Notation
- Lightweight
- Easy to understand
- Not specific to languages



JSON Structure

Simple JSON format

```
{
  “name”: "Babu",
  “age”: 45,
  “city”: "Chennai"
}
```

Simple JSON Array

```
{
  “name”: "Babu",
  “age”: 45,
  “companies”: ["Syntel", "HCL", "HP", "TL"]
}
```

JSON Nested Arrays

```
{
  "name":"Babu",
  "age":45,
  "companies":[
     {"name" : "syntel", "years" : "3.1", "customers" : 4, "country" : ["india", "usa"]},
     {"name" : "hcl", "years" : "1", "customers" : 1, "country" : ["india"]},
     {"name" : "hp", "years" : "13.2", "customers" : 18, "country" : ["india",
"usa","uk","sg"]},
     {"name" : "tl", "years" : "4.3", "customers" : 8, "country" : ["india", "usa"]}
   ]
}
```

XML Structure

Simple XML

```xml
<?xml version="1.0" encoding="UTF-8"?>
<details>
    <person>
        <name>Babu</name>
        <age>45</age>
        <city>chennai</city>
     </person>
</details>
```

5 Step Process

Add Request

Headers + Authorization + Request Query / body

End Point

Server + Port + Resources

Send Request

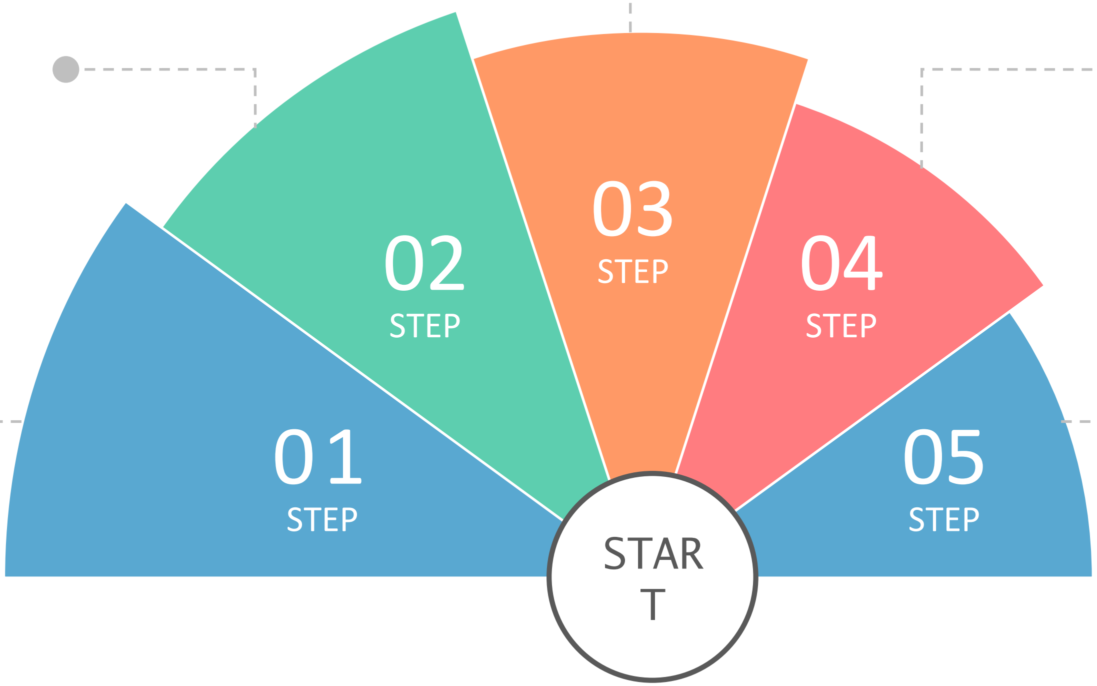
Send to the server with the appropriate request type

Requirement

Swagger / PostMan Documentation

Validate Response

Verify with status code and body



Documentation Tool

VS

https://covid-19-apis.postman.com/

Thank You!

Testleaf
Always Ahead